Tarefa 11 - Particionamento de dados e balanceamento de carga

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

A atividade consiste em simular o movimento de um fluido em 2D com a equação de Navier–Stokes, considerando apenas a viscosidade (sem pressão e sem forças externas), e depois paralelizar o código com OpenMP. O objetivo principal é observar como o **particionamento dos dados** entre as threads e o **balanceamento de carga**, controlados pelas cláusulas `schedule` e `collapse`, afetam o desempenho. A medição foi feita com o PaScal Suite.

2 Simulação do fluido

O código foi validado antes de ser paralelizado, com dois testes.

- **Fluido parado**: o campo de velocidade deve continuar nulo. A Figura 1 mostra que isso aconteceu: nos quatro instantes exibidos, o campo é zero em toda a malha.

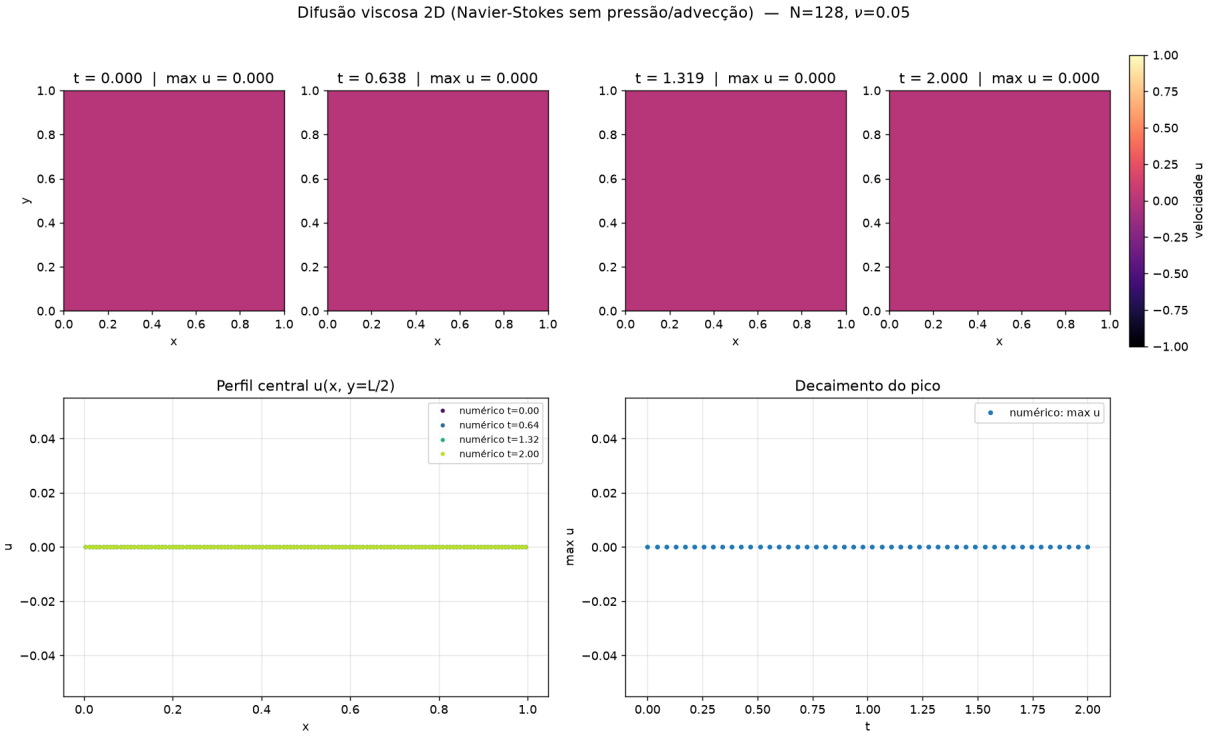


Figura 1: Fluido parado: o campo permanece nulo.

- **Perturbação no fluido**: uma região de velocidade mais alta no centro deve se espalhar suavemente. A Figura 2 mostra o pico diminuindo (de 0,994 para 0,200) e a região se alargando, sem oscilações. O resultado numérico acompanha a solução analítica traçada nos gráficos.

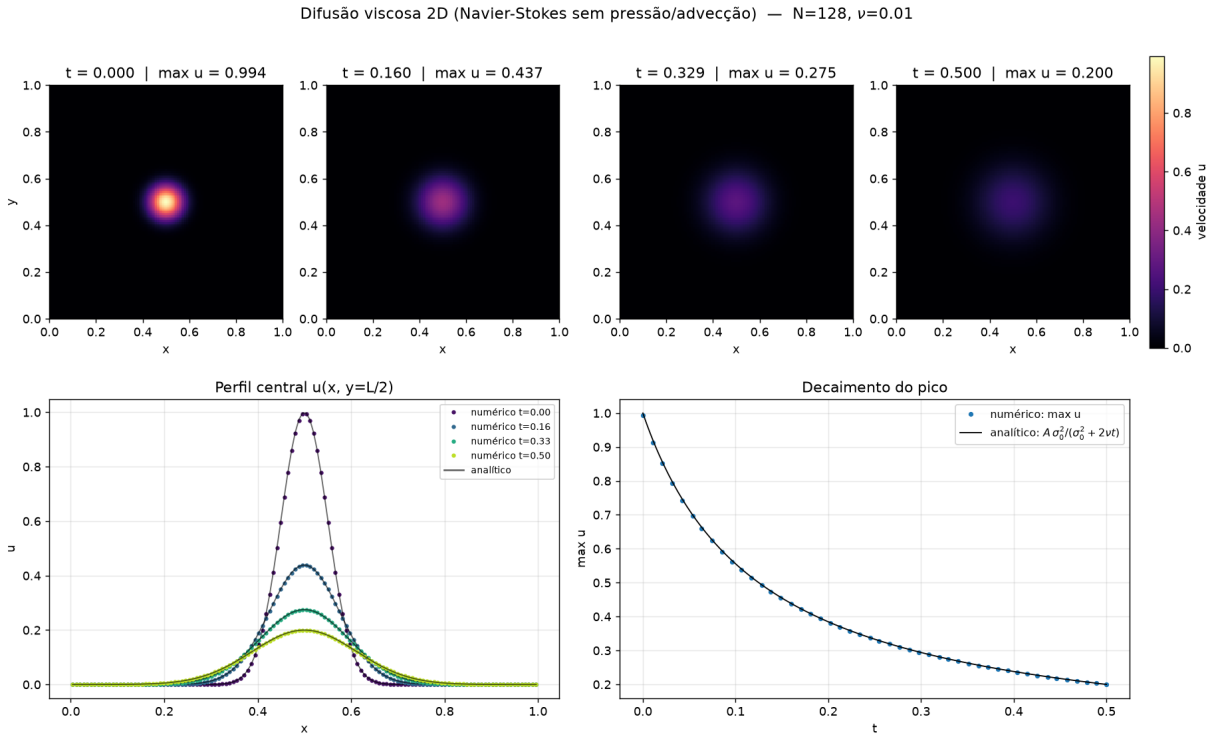


Figura 2: Difusão da perturbação, comparada com a solução analítica.

Nos dois casos o comportamento foi o esperado, e o código foi considerado validado para os testes de desempenho.

3 Paralelização com OpenMP

3.1 Particionamento de dados e balanceamento de carga

O trabalho do programa é aplicar o mesmo cálculo a cada ponto de uma malha 1024×1024, a cada passo de tempo. Paralelizar significa dividir esses pontos entre as threads. Foram avaliadas duas formas de dividir:

- `schedule(static)`: o laço externo (as linhas da malha) é dividido em blocos contíguos de mesmo tamanho, um por thread, definidos antes da execução. Com 8 threads, cada uma recebe 128 das 1024 linhas.

- `collapse(2)`: os dois laços (linhas e colunas) são tratados como um único conjunto de 1024^2 pontos, e esse conjunto é dividido entre as threads, também de forma estática. Com 8 threads, cada uma recebe 131.072 pontos.

Tabela 1: Como cada versão divide o trabalho entre as threads.

Versão	O que é dividido	Unidades	Por thread (8 threads)
`schedule(static)`	linhas da malha	1.024	128 linhas
`collapse(2)`	pontos da malha	1.048.576	131.072 pontos

Balanceamento de carga. Todos os pontos exigem o mesmo trabalho, e 1024 é divisível por 2, 4 e 8. Por isso, as duas versões distribuem o trabalho de forma igual entre as threads em todas as configurações testadas, e não existe desbalanceamento a corrigir. Esse é o cenário em que o escalonamento estático é indicado. As políticas `dynamic` e `guided`, que entregam pedaços de trabalho às threads durante a execução, são úteis quando o custo de cada iteração varia, o que não ocorre aqui.

O que o `collapse` muda no código. Para fundir os laços, o cálculo dos vizinhos periódicos das linhas (`im` e `ip`) precisou ser movido para dentro do laço interno, e passa a ser refeito a cada ponto (Apêndice A). A versão com `collapse` divide o trabalho em partes menores, mas executa mais instruções por ponto.

3.2 Configuração do experimento

A malha foi fixada em 1024×1024, e o único parâmetro de entrada foi o tempo final simulado. As entradas i1 a i4 dobram o tempo simulado a cada passo, de modo que o trabalho total também dobra. O PaScal Analyzer foi usado com instrumentação manual da região do passo de tempo, para 1, 2, 4 e 8 cores.

3.3 Resultados

A Tabela 2 traz os tempos de execução, e a Tabela 3 traz o speedup e a eficiência médios, ambos do PaScal Viewer. Nesses dois indicadores, cada versão é comparada com a sua própria execução com 1 core. A Figura 3 mostra os mapas do Viewer; as escalas de cores de cada mapa são diferentes.

Tabela 2: Tempo de execução (s) por número de cores e entrada.

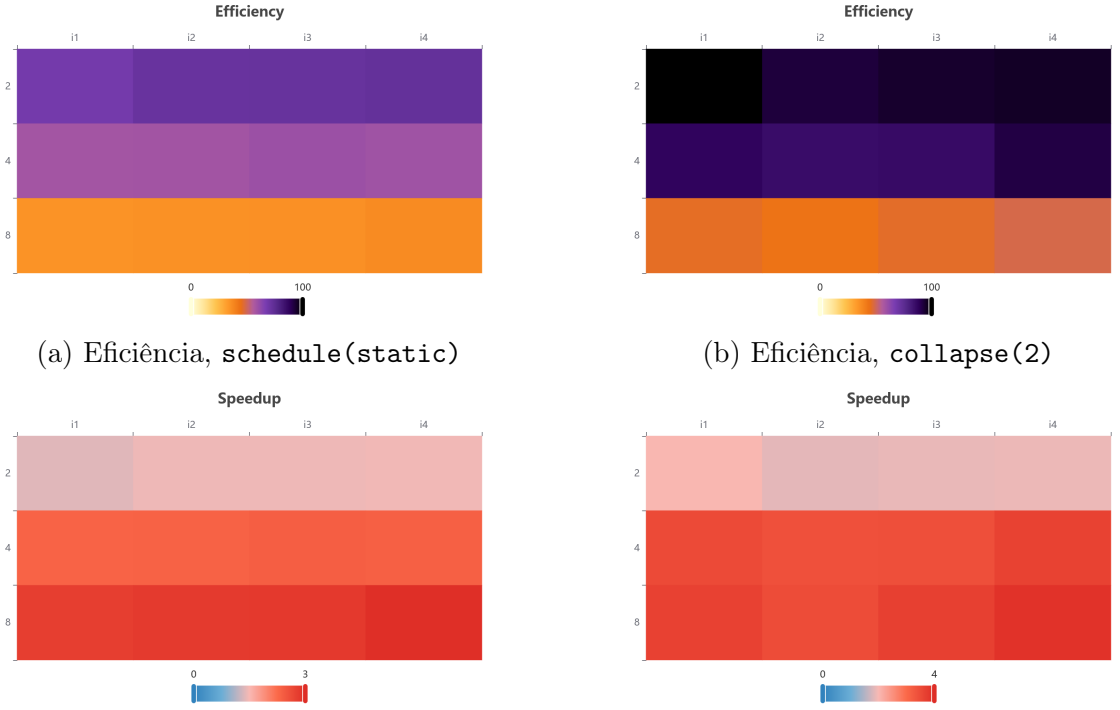
	`schedule(static)`				`collapse(2)`			
Cores	i1	i2	i3	i4	i1	i2	i3	i4
1	0,741	1,486	3,008	5,878	1,115	2,107	4,217	8,379
2	0,541	1,032	2,084	4,021	0,549	1,139	2,233	4,396
4	0,317	0,632	1,252	2,478	0,317	0,617	1,224	2,288
8	0,265	0,523	1,055	1,980	0,304	0,603	1,143	2,145

3.4 Diferenças de desempenho

Pelo speedup e pela eficiência, o `collapse` parece melhor. Com 2, 4 e 8 cores, o `collapse` tem speedup maior e eficiência maior (Tabela 3). Com 2 cores, a eficiência média é de 95,9%, contra 71,4% do `static`.

Tabela 3: Speedup e eficiência médios (média das quatro entradas), cada versão em relação ao seu próprio 1 core.

	Speedup		Eficiência (%)	
Cores	`static`	`collapse`	`static`	`collapse`
2	1,43	1,92	71,4	95,9
4	2,37	3,51	59,2	87,8
8	2,86	3,69	35,8	46,1



(a) Eficiência, `schedule(static)`

(b) Eficiência, `collapse(2)`

(c) Speedup, `schedule(static)`

(d) Speedup, `collapse(2)`

Figura 3: Mapas do PaScal Viewer (cores × entradas).

Mas esses indicadores usam bases diferentes. Com 1 core, o `collapse` leva em média 44% mais tempo que o `static` (na maior entrada, 8,379 s contra 5,878 s). Como o speedup compara cada versão com o próprio tempo com 1 core, o `collapse` parte de uma base mais lenta, o que melhora seus indicadores. Para comparar de forma justa, a Tabela 4 mostra o tempo do `collapse` em relação ao do `static`, e o speedup das duas versões usando o 1 core do `static` como referência.

Em tempo absoluto, o `static` é equivalente ou melhor.

- Com 2 cores, o `collapse` é cerca de 7% mais lento.
- Com 4 cores, os tempos são praticamente iguais (o `collapse` é cerca de 3% mais rápido, até 7,7% na maior entrada). Os tempos não trazem desvio padrão, então uma diferença desse tamanho pode ser variação entre execuções.
- Com 8 cores, o `collapse` é cerca de 12% mais lento. Em todas as entradas, o menor tempo é o do `static` com 8 cores.

Tabela 4: Comparação em tempo absoluto (média das quatro entradas). O speedup usa o 1 core do `static` como referência para as duas versões.

	Tempo	Speedup (referência comum)	
Cores	`collapse` / `static`	`static`	`collapse`
1	1,44	1,00	0,70
2	1,07	1,43	1,34
4	0,97	2,37	2,44
8	1,12	2,86	2,57

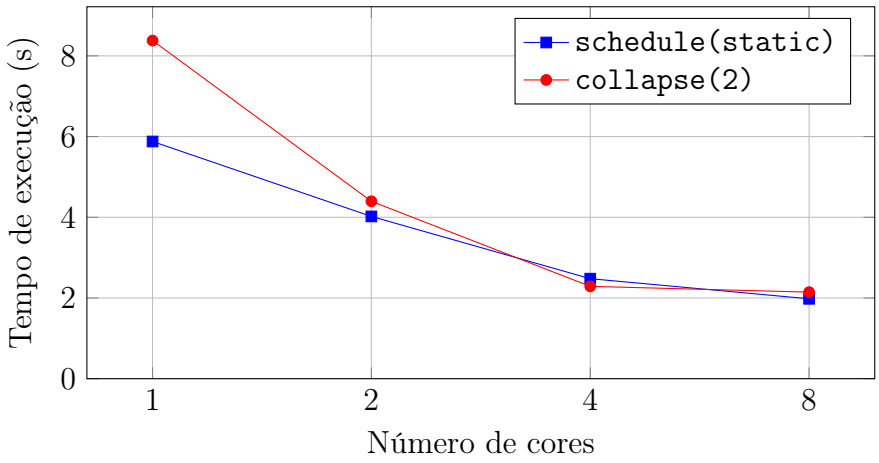


Figura 4: Tempo de execução na maior entrada (i4).

Ganho ao passar de 4 para 8 cores. O `static` reduz o tempo em 16% a 20%, enquanto o `collapse` reduz apenas 2% a 7%. Nas duas versões, a eficiência com 8 cores é baixa (35,8% e 46,1%)

Sobre o balanceamento. Como as duas versões já dividem o trabalho de forma igual entre as threads, a diferença de desempenho não vem do balanceamento de carga. Dividir o trabalho em partes menores, como faz o `collapse`, não trouxe ganho neste problema, e a versão com `collapse` executa mais instruções por ponto.

4 Conclusão

- A simulação teve o comportamento esperado: o fluido parado permaneceu parado, e a perturbação se espalhou suavemente, de acordo com a solução analítica.
- Como a carga é uniforme e a malha é divisível pelo número de threads, o escalonamento estático já balanceia a carga, e o `schedule(static)` no laço externo foi suficiente.
- O `collapse(2)` aparenta escalar melhor no PaScal Viewer, mas isso se deve em boa parte ao seu tempo com 1 core ser 44% maior.
- Em tempo absoluto, o `schedule(static)` é equivalente ou melhor, e dá o menor tempo com 8 cores (1,980 s contra 2,145 s na maior entrada).

- Speedup e eficiência relativos ao próprio 1 core devem ser lidos junto com os tempos absolutos antes de se escolher uma cláusula.

A Código-fonte dos kernels

Os trechos a seguir são o passo de tempo das duas versões avaliadas. A compilação foi feita com `gcc -O2 -Wall -fopenmp ... -lm -lmpascalops`.

```c
#define IDX(i, j, N) ((i) * (N) + (j))

static void passo(const double *u, const double *v,
                  double *un, double *vn, int N, double r)
{
    pascal_start(1);
    #pragma omp parallel for schedule(runtime)
    for (int i = 0; i < N; i++) {
        int im = (i == 0)     ? N - 1 : i - 1;
        int ip = (i == N - 1) ? 0     : i + 1;
        for (int j = 0; j < N; j++) {
            int jm = (j == 0)     ? N - 1 : j - 1;
            int jp = (j == N - 1) ? 0     : j + 1;

            double lap_u = u[IDX(ip, j, N)] + u[IDX(im, j, N)]
                         + u[IDX(i, jp, N)] + u[IDX(i, jm, N)]
                         - 4.0 * u[IDX(i, j, N)];
            double lap_v = v[IDX(ip, j, N)] + v[IDX(im, j, N)]
                         + v[IDX(i, jp, N)] + v[IDX(i, jm, N)]
                         - 4.0 * v[IDX(i, j, N)];

            un[IDX(i, j, N)] = u[IDX(i, j, N)] + r * lap_u;
            vn[IDX(i, j, N)] = v[IDX(i, j, N)] + r * lap_v;
        }
    }
    pascal_stop(1);
}
```

Listing 1: Versão `schedule`: paralelismo apenas no laço externo

```c
static void passo(const double *u, const double *v,
                  double *un, double *vn, int N, double r)
{
    pascal_start(1);
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int im = (i == 0)     ? N - 1 : i - 1;
            int ip = (i == N - 1) ? 0     : i + 1;
            int jm = (j == 0)     ? N - 1 : j - 1;
            int jp = (j == N - 1) ? 0     : j + 1;

            double lap_u = u[IDX(ip, j, N)] + u[IDX(im, j, N)]
                         + u[IDX(i, jp, N)] + u[IDX(i, jm, N)]
                         - 4.0 * u[IDX(i, j, N)];
            double lap_v = v[IDX(ip, j, N)] + v[IDX(im, j, N)]
                         + v[IDX(i, jp, N)] + v[IDX(i, jm, N)]
                         - 4.0 * v[IDX(i, j, N)];

            un[IDX(i, j, N)] = u[IDX(i, j, N)] + r * lap_u;
            vn[IDX(i, j, N)] = v[IDX(i, j, N)] + r * lap_v;
        }
    }
```

```
24      pascal_stop(1);
25  }
```

Listing 2: Versão `collapse`: laços `i` e `j` fundidos